



School of Engineering and Technology

Department of Computer Science and Engineering

**PREPARED BY
PROF.**

LAB MANUAL

COURSE NAME	CYBER SECURITY LAB
COURSE CODE	CS385
YEAR	3RD,B.TECH
SEMESTER	5TH
SESSION	2026-2027



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CYBER SECURITY LAB

DEPARTMENT VISION & MISSION

VISION

To become a Center of Excellence in education and research in the field of Computer Science & Engineering, create conducive environment and excellent facilities befitting to the need of learning and producing technical competent Computer Engineer and Information Technology professionals by imparting sound technical skills and profound knowledge.

MISSION

1. M1: To impart curricular and co-curricular technical knowledge to the students to turn out competent and well-groomed engineers.
2. M2: To provide students with a strong theoretical and practical background across the computer science discipline with an emphasis on software development.
3. M3: To promote academic growth of students and faculty members by acquiring new technical knowledge in research leading to innovative projects.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CYBER SECURITY LAB

Recommended Pre-requisite: Computer Networks, Basic python programming

COURSEOUTCOMES

After going through this course, the student will be able to

CO1	Develop Python scripts for fundamental cyber security tasks such as file handling, network communication, and data extraction from websites.
CO2	Implement and demonstrate basic cryptographic techniques, hashing algorithms, and encryption/decryption schemes using Python.
CO3	Apply Python tools and libraries to analyze network traffic, perform penetration testing, and interact with security-focused APIs.
CO4	Design and develop basic security applications including malware analysis tools and intrusion detection systems using Python.

PROGRAMME OUTCOMES

- **PO1:** Engineering Knowledge: Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
- **PO2:** Problem Analysis: Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
- **PO3:** Design/ Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.
- **PO4:** Conduct investigations of complex problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.
- **PO5:** The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
- **PO6:** Environment and Sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.
- **PO7:** Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
- **PO8:** Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.
- **PO9:** Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.
- **PO10:** Project Management and Finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO11:** Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAMME SPECIFIC OUTCOMES

- **PSO1:** Make the students industry ready by providing the required training in cutting edge technology
- **PSO2:** The ability to employ core computing concepts and optimization approaches to create more efficient and effective computing systems.
- **PSO3:** The Ability to apply their innovative and research-oriented skills in the field of networking, web design, cloud computing, IoT, machine learning, blockchain technology and data analytics.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- **PEO1:** Excel in professional career by acquiring knowledge in mathematical, basic science, computing and engineering principles.
- **PEO2:** The graduates should be able to use their skills with a strong base to prepare them for higher learning and research activities.
- **PEO3:** To develop an ability to analyze the requirements, understand the technical specifications, design and provide novel engineering solutions and produce efficient product designs.
- **PEO4:** The graduate should exhibit professionalism, ethical attitude, communication skills, team work in their profession.

COURSE OBJECTIVE

- To develop fundamental Python programming skills for implementing various cyber security applications and automation tasks.
- To provide practical knowledge of file handling, string manipulation, and network communication using Python for security-related tasks.
- To enable students to implement cryptographic techniques, including encryption, decryption, and hashing algorithms, for securing digital information.
- To develop hands-on experience in building client-server applications and analyzing network traffic using Python-based networking libraries and tools.
- To introduce web scraping, penetration testing, and the use of security-related APIs for information gathering and vulnerability assessment.
- To provide practical exposure to basic static malware analysis techniques, including file signature analysis and string extraction using Python.
- To develop the ability to design and implement basic cyber security solutions, such as intrusion detection systems, while strengthening analytical and problem-solving skills in cyber security.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CYBER SECURITY LAB
CONTENTS

SL.NO.	LIST OF EXPERIMENTS	PAGE NO.
EXPERIMENT:1		
1.	Writing simple Python scripts for tasks like string manipulation, reading from and writing to files, basic network communication.	9-12
EXPERIMENT:2		
2.	Implementing basic encryption and decryption algorithms in Python Caesar cipher, AES, DES	13-16
EXPERIMENT:3		
3.	Using python to generate and verify hashes (MD5, SHA-256) for files	17-20
EXPERIMENT:4		
4.	Building a simple Python Client-Server application, understanding sockets.	21-23
EXPERIMENT:5		
5.	Writing a python script to capture and analyze network packets(using libraries like Scapy or PySpark)	24-25
EXPERIMENT:6		
6.	Creating a web scraper in Python to gather data from websites(using BeautifulSoup, Selenium)	26-27
EXPERIMENT:7		
7.	Simple penetration testing tasks using Python (Eg: port scanning, vulnerability scanning with tools like Nmap in Python.	28-29
EXPERIMENT:8		
8.	Using python to interact with security-related APIs (eg. VirusTotal, Shodan)	30-31
EXPERIMENT:9		
9.	Writing python scripts for basic static malware analysis (file signature analysis, string extraction).	32-33

EXPERIMENT NO.-01

1. Aim of the Experiment:

To write simple Python scripts for string manipulation, file handling, and basic network communication.

2. Objective:

- To understand basic Python programming concepts.
- To perform string manipulation operations.
- To implement file reading and writing operations.
- To understand basic network communication using Python socket programming.

3. Problem Statement:

Develop a Python program that accepts runtime input from the user to perform string manipulation, read and write data into a text file, and retrieve the IP address of a website using socket programming. The program should demonstrate basic cyber security programming concepts useful in automation and networking.

4. Algorithm:

Algorithm for String Manipulation

- Step 1:** Start
- Step 2:** Read a string from the user.
- Step 3:** Display the original string.
- Step 4:** Convert the string into uppercase.
- Step 5:** Convert the string into lowercase.
- Step 6:** Reverse the string.
- Step 7:** Display the string length.
- Step 8:** Stop.

Algorithm for File Handling

- Step 1:** Read the filename.
- Step 2:** Read text from the user.
- Step 3:** Create the file.

Step 4: Write the text into the file.

Step 5: Read the contents of the file.

Step 6: Display the file contents.

Algorithm for Network Communication

Step 1: Read the website name.

Step 2: Resolve its IP address.

Step 3: Display the IP address.

Step 4: Stop.

5.Code Implementation:

String Manipulation

```
import socket
print("\n--- STRING MANIPULATION ---")
text = input("Enter a string: ")
print("Original String :", text)
print("Uppercase      :", text.upper())
print("Lowercase      :", text.lower())
print("Reversed String :", text[::-1])
print("Length         :", len(text))
```

File Read and Write

```
import socket
print("\n--- FILE HANDLING ---")
filename = input("Enter file name: ")
content = input("Enter text to store in file: ")
with open(filename, "w") as file:
    file.write(content)
print(f"Data successfully written to {filename}")
with open(filename, "r") as file:
    data = file.read()
print("Content read from file:")
print(data)
```

Basic Network Communication

```
import socket
print("\n--- NETWORK COMMUNICATION ---")
website = input("Enter website: ")
try:
    ip_address = socket.gethostbyname(website)
    print(f"Website : {website}")
    print(f"IP Address : {ip_address}")
except Exception as e:
    print("Error:", e)
```

6. Execution and Output Discussion:

STRING MANIPULATION

INPUT:

Enter a string: cyber security

OUTPUT:

Original String : cyber security

Uppercase : CYBER SECURITY

Lowercase : cyber security

Reversed String : ytiruces rebyc

Length : 14

FILE HANDLING

INPUT:

Enter file name: sample.txt

Enter text to store in file: hii Priya

OUTPUT:

Data successfully written to
sample.txt

Content read from file:

hii priya

NETWORK COMMUNICATION

Input

Enter website: instagram.com

Output

Website : instagram.com

IP Address : 57.144.74.34

The program successfully performs three fundamental Python operations. The string manipulation section demonstrates text processing functions. The file handling section stores and retrieves information from a file. The networking section resolves a domain name into its IP address using the socket library, introducing students to basic network programming concepts.

7. Conclusion:

The experiment successfully demonstrated the implementation of Python programs for string manipulation, file handling, and basic network communication. These concepts provide the foundation for developing automation scripts and cyber security applications using Python.

EXPERIMENT NO.-02

1. Aim of the Experiment:

To implement basic encryption and decryption algorithms in Python using the **Caesar Cipher, AES, and DES** algorithms.

2. Objective:

- To understand the fundamentals of symmetric encryption techniques.
- To implement the Caesar Cipher encryption and decryption algorithm.
- To perform AES encryption and decryption using Python.
- To implement DES encryption and decryption using Python.
- To compare classical and modern encryption techniques.
- To understand the importance of encryption in protecting confidential information.
- To gain hands-on experience with Python cryptography libraries.

3. Problem Statement:

Data transmitted over communication networks is vulnerable to unauthorized access. Encryption algorithms protect sensitive information by converting plaintext into cipher text. Develop Python programs to implement the Caesar Cipher, AES, and DES algorithms for encrypting and decrypting user-provided messages.

4. Algorithm:

Algorithm for Caesar Cipher

Step 1: Start.

Step 2: Read the plaintext from the user.

Step 3: Read the encryption key (shift value).

Step 4: Encrypt each alphabet by shifting it according to the key.

Step 5: Display the encrypted message.

Step 6: Decrypt the ciphertext using the same key.

Step 7: Display the original message.

Step 8: Stop.

Algorithm for AES Encryption

Step 1: Start.

Step 2: Read the plaintext.

Step 3: Generate a random AES key.
Step 4: Encrypt the plaintext using AES.
Step 5: Display the ciphertext.
Step 6: Decrypt the ciphertext.
Step 7: Display the original message.
Step 8: Stop.

Algorithm for DES Encryption

Step 1: Start.
Step 2: Read the plaintext.
Step 3: Generate a DES key.
Step 4: Encrypt the plaintext.
Step 5: Display the encrypted message.
Step 6: Decrypt the ciphertext.
Step 7: Display the original message.
Step 8: Stop.

5. Code Implementation:

```
!pip install pycryptodome
print("==== Caesar Cipher =====")
text = input("Enter Plain Text : ")
shift = int(input("Enter Shift Key : "))
encrypted = ""
for ch in text:
    if ch.isalpha():
        if ch.isupper():
            encrypted += chr((ord(ch)-65+shift)%26+65)
        else:
            encrypted += chr((ord(ch)-97+shift)%26+97)
    else:
        encrypted += ch

print("\nEncrypted Text :", encrypted)
decrypted = ""
for ch in encrypted:
    if ch.isalpha():
        if ch.isupper():
            decrypted += chr((ord(ch)-65-shift)%26+65)
        else:
```

```

        decrypted += chr((ord(ch)-97-shift)%26+97)
    else:
        decrypted += ch
print("Decrypted Text :", decrypted)

```

Code for AES Encryption

```

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
import base64
print("=====  

AES Encryption =====")
message = input("Enter Message : ")
key = get_random_bytes(16)
cipher = AES.new(key, AES.MODE_EAX)
ciphertext, tag = cipher.encrypt_and_digest(message.encode())
print("\nEncrypted Text:")
print(base64.b64encode(ciphertext).decode())
cipher = AES.new(key, AES.MODE_EAX, nonce=cipher.nonce)
plaintext = cipher.decrypt(ciphertext)
print("\nDecrypted Text:")
print(plaintext.decode())

```

Code for DES Encryption

```

from Crypto.Cipher import DES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import base64
print("=====  

DES Encryption =====")
message = input("Enter Message : ")
key = get_random_bytes(8)
cipher = DES.new(key, DES.MODE_ECB)
ciphertext = cipher.encrypt(pad(message.encode(),8))
print("\nEncrypted Text:")
print(base64.b64encode(ciphertext).decode())
cipher = DES.new(key, DES.MODE_ECB)
plaintext = unpad(cipher.decrypt(ciphertext),8)
print("\nDecrypted Text:")
print(plaintext.decode())

```

6.Execution and Output Discussion:

Caesar Cipher

INPUT:

Enter Plain Text : priya

Enter Shift Key : 3

OUTPUT:

Encrypted Text : sulbd

Decrypted Text : priya

AES Encryption

INPUT:

Enter Message : i love Priya

OUTPUT:

Encrypted Text:

xAi0oneOdXRix1NW

Decrypted Text:

i love priya

DES Encryption

INPUT:

Enter Message : i love bunny

OUTPUT:

Encrypted Text:

Tzdck1E3VE/y1o86jz6wyg==

Decrypted Text:

i love bunny

The experiment successfully implemented the **Caesar Cipher, AES, and DES** algorithms in Python. It demonstrated the encryption and decryption of messages and highlighted the differences between classical and modern encryption techniques.

7. Conclusion:

The experiment successfully implemented the **Caesar Cipher, AES, and DES** algorithms using Python. It demonstrated encryption and decryption techniques and provided practical knowledge of symmetric cryptography for securing data.

1. Aim of the Experiment:

To generate and verify cryptographic hashes (MD5 and SHA-256) for text messages and files using Python.

2. Objective:

- To understand cryptographic hash functions.
- To generate MD5 and SHA-256 hashes for messages and files.
- To verify data integrity using hash values.
- To compare MD5 and SHA-256 algorithms.
- To gain practical experience using Python's hashlib library.

3. Problem Statement:

Hashing algorithms are widely used in cyber security to ensure data integrity and verify file authenticity. Any modification to the original data produces a completely different hash value. Develop a Python program that generates and verifies MD5 and SHA-256 hashes for user-entered messages and files.

4. Algorithm:

Algorithm for Hashing a Message

Step 1: Start.

Step 2: Read a message from the user.

Step 3: Generate the MD5 hash.

Step 4: Generate the SHA-256 hash.

Step 5: Display both hash values.

Step 6: Stop.

Algorithm for Hashing a File

Step 1: Start.

Step 2: Read the filename from the user.

Step 3: Open the file in binary mode.

Step 4: Read the file contents.

Step 5: Generate the MD5 hash.

Step 6: Generate the SHA-256 hash.

Step 7: Display both hash values.

Step 8: Stop.

Algorithm for Hash Verification

Step 1: Generate the original hash.

Step 2: Read the verification hash from the user.

Step 3: Compare both hashes.

Step 4: If equal, display **Integrity Verified**.

Step 5: Otherwise display **File/Message Modified**.

Step 6: Stop.

5.Code Implementation:

Code For Hash Generation For Message

```
import hashlib
print("==== HASH GENERATION FOR MESSAGE ====")
message = input("Enter a message: ")
md5_hash = hashlib.md5(message.encode()).hexdigest()
sha256_hash = hashlib.sha256(message.encode()).hexdigest()
print("\nMD5 Hash:")
print(md5_hash)
print("\nSHA-256 Hash:")
print(sha256_hash)
```

Code For Hash Generation For File

```
import hashlib
print("==== HASH GENERATION FOR FILE ====")
filename = input("Enter file name: ")
try:
    with open(filename, "rb") as file:
        data = file.read()
        md5_hash = hashlib.md5(data).hexdigest()
        sha256_hash = hashlib.sha256(data).hexdigest()
        print("\nMD5 Hash:")
        print(md5_hash)
        print("\nSHA-256 Hash:")
        print(sha256_hash)
except FileNotFoundError:
    print("File not found!")
```

Code For Hash Generation For File

```
import hashlib
print("==== HASH VERIFICATION ====")
message = input("Enter Original Message: ")
original_hash = hashlib.sha256(message.encode()).hexdigest()
```

```
print("\nGenerated SHA-256 Hash:")
print(original_hash)
verify_hash = input("\nEnter hash to verify: ")
if original_hash == verify_hash:
    print("\nIntegrity Verified")
else:
    print("\nIntegrity Check Failed")
```

6.Execution and Output Discussion:

HASH GENERATION FOR MESSAGE

INPUT:

Enter a message: Cyber Security Lab

OUTPUT:

MD5 Hash:

45468b4cf86546498c64a21941c97871

SHA-256 Hash:

27f7ebbc586b4c992f5df0163094b3e54041edd9c7a0721b64931f8c2c42c31

HASH GENERATION FOR FILE

INPUT:

Enter file name: priya.txt

OUTPUT:

MD5 Hash:

4c42e607d529e0d59f4474b8c94599b8

SHA-256 Hash:

dbe8bbcd3db3d9d4dcb5d6f8b0f06d8d7cf896f88a7b5d6d687eecfe0f48d998

HASH VERIFICATION

INPUT:

Enter Original Message: Cyber Security Lab

OUTPUT:

Generated SHA-256 Hash:

27f7ebbc586b4c992f5df0163094b3e54041edd9c7a0721b64931f8c2c42c31

Enter hash to verify:

27f7ebbc586b4c992f5df0163094b3e54041edd9c7a0721b64931f8c2c42c31

Integrity Verified

The program successfully generated **MD5** and **SHA-256** hash values for messages and files. It demonstrated how hash functions are used to verify data integrity and detect modifications.

7. Conclusion:

The experiment successfully demonstrated the generation and verification of **MD5** and **SHA-256** hashes using Python. It provided practical knowledge of cryptographic hashing and its role in ensuring data integrity and security.

EXPERIMENT NO.-04

1. Aim of the Experiment:

To build a simple client-server application in Python using socket programming for establishing communication between two systems.

2. Objective:

- To understand the basics of socket programming.
- To establish client-server communication using Python.
- To implement a TCP-based client-server application.
- To exchange messages between client and server.
- To gain practical experience in network programming.

3. Problem Statement:

Communication between computers in a network is achieved through sockets. Develop a Python client-server application that establishes a TCP connection between a server and a client. The server should receive messages from the client and send appropriate responses, demonstrating the fundamentals of network communication.

4. Algorithm:

Step 1: Start.

Step 2: Import the required modules (socket, threading, and time).

Step 3: Define the host address and port number.

Step 4: Create a server socket, bind it to the host and port, and start listening for client connections.

Step 5: Create a client socket and connect it to the server.

Step 6: Accept the client connection on the server.

Step 7: Read a message from the client and send it to the server.

Step 8: Receive and display the client's message on the server.

Step 9: Read a reply from the server and send it back to the client.

Step 10: Receive and display the server's reply on the client.

Step 11: Close both the client and server sockets.

Step 12: Stop.

5.Code Implementation:

```
import socket
import threading
import time
HOST = "127.0.0.1"
PORT = 5050
def server_program():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print("Server is waiting for connection...")
    conn, addr = server.accept()
    print("Connected by:", addr)
    data = conn.recv(1024).decode()
    print("Message received from Client:", data)
    reply = input("Enter reply from Server: ")
    conn.send(reply.encode())
    conn.close()
    server.close()
def client_program():
    time.sleep(1)    # Wait until server starts
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((HOST, PORT))
    message = input("Enter message from Client: ")
    client.send(message.encode())
    reply = client.recv(1024).decode()
    print("Reply from Server:", reply)
    client.close()
server_thread = threading.Thread(target=server_program)
client_thread = threading.Thread(target=client_program)
server_thread.start()
client_thread.start()
server_thread.join()
client_thread.join()
print("\nCommunication Successful!")
```

6.Execution and Output Discussion:

INPUT:

Enter message from Client: Hello Server

Enter reply from Server: Hello Client

OUTPUT:

Server is waiting for connection...

Connected by: ('127.0.0.1', 50872)

Message received from Client: Hello Server

Reply from Server: Hello Client

Communication Successful!

The program successfully established a TCP connection between the client and the server. The client sent a message to the server, and the server responded successfully, demonstrating basic socket-based communication.

7. Conclusion:

The experiment successfully demonstrated client-server communication using Python socket programming. It provided practical understanding of TCP sockets and message exchange between two network applications.

EXPERIMENT NO.-05

1. Aim of the Experiment:

To capture and analyze network packets using Python and the Scapy library.

2. Objective:

- To understand packet capturing using Scapy.
- To capture network packets.
- To analyze packet details such as source, destination, and protocol.
- To gain practical experience in network packet analysis.

3. Problem Statement:

Develop a Python program using the Scapy library to capture a specified number of network packets and display their basic information for network analysis.

4. Algorithm:

Step 1: Start.

Step 2: Import the Scapy library.

Step 3: Read the number of packets to capture.

Step 4: Capture the packets.

Step 5: Display the source IP, destination IP, and protocol of each packet.

Step 6: Stop.

5. Code Implementation:

```
!pip install scapy
from scapy.all import sniff
from scapy.layers.inet import IP
print("=== Packet Capture and Analysis ===")
count = int(input("Enter number of packets to capture: "))
def packet_info(packet):
    if packet.haslayer(IP):
        print("\nSource IP      :", packet[IP].src)
        print("Destination IP :", packet[IP].dst)
        print("Protocol      :", packet[IP].proto)
print("\nCapturing packets...\n")
sniff(prn=packet_info, count=count)
print("\nPacket Capture Completed.")
```

6.Execution and Output Discussion:

INPUT:

Enter number of packets to capture: 3

OUTPUT:

Capturing packets...

Source IP : 192.168.1.10

Destination IP : 142.250.183.110

Protocol : 6

Source IP : 192.168.1.10

Destination IP : 8.8.8.8

Protocol : 17

Source IP : 192.168.1.10

Destination IP : 172.217.160.46

Protocol : 6

Packet Capture Completed.

The program successfully captured the specified number of network packets and displayed their source IP, destination IP, and protocol information using the Scapy library.

7. Conclusion:

The experiment successfully demonstrated network packet capturing and analysis using Python and Scapy, providing practical knowledge of packet inspection and network traffic analysis.

EXPERIMENT NO.-06

1. Aim of the Experiment:

To develop a Python program that extracts data from a website using the BeautifulSoup library.

2. Objective:

- To understand the basics of web scraping.
- To fetch webpage content using Python.
- To extract HTML elements using BeautifulSoup.
- To gain practical experience in web data extraction.

3. Problem Statement:

Develop a Python program that accepts a website URL as input, retrieves the webpage content, and extracts the page title and all headings using the BeautifulSoup library.

4. Algorithm:

Step 1: Start.

Step 2: Import the required libraries.

Step 3: Read the website URL from the user.

Step 4: Fetch the webpage content.

Step 5: Parse the HTML using BeautifulSoup.

Step 6: Extract the page title and headings.

Step 7: Display the extracted data.

Step 8: Stop.

5. Code Implementation:

```
!pip install beautifulsoup4 requests
import requests
from bs4 import BeautifulSoup
print("==== WEB SCRAPER =====")
url = input("Enter Website URL: ")
try:
    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")
```

```
print("\nWebsite Title:")
print(soup.title.string)
print("\nHeadings Found:\n")
headings = soup.find_all(["h1", "h2", "h3"])
if headings:
    for i, heading in enumerate(headings, start=1):
        print(f" {i}. {heading.get_text(strip=True)}")
else:
    print("No headings found.")
except Exception as e:
    print("Error:", e)
```

6.Execution and Output Discussion:

INPUT:

Enter Website URL: <https://instagram.com>

OUTPUT:

Website Title: Instagram

Headings Found: No headings found.

The program successfully fetched the webpage and extracted the page title and headings using the BeautifulSoup library.

7. Conclusion:

The experiment successfully demonstrated web scraping using Python and BeautifulSoup. It provided practical knowledge of extracting useful information from web pages.

EXPERIMENT NO.-07

1. Aim of the Experiment:

To develop a Python program that performs port scanning on a target host to identify open ports.

2. Objective:

- To understand the basics of port scanning.
- To identify open ports on a target host.
- To gain practical experience in penetration testing using Python.
- To understand the role of port scanning in network security.

3. Problem Statement:

Develop a Python program that accepts a target host and a range of port numbers as input, scans the specified ports, and displays whether each port is open or closed.

4. Algorithm:

Step 1: Start.

Step 2: Import the socket module.

Step 3: Read the target host, starting port, and ending port.

Step 4: Scan each port in the specified range.

Step 5: Display whether the port is open or closed.

Step 6: Stop.

5. Code Implementation:

```
import socket
print("==== PORT SCANNER =====")
host = input("Enter Target Host (Example: scanme.nmap.org): ")
start_port = int(input("Enter Starting Port: "))
end_port = int(input("Enter Ending Port: "))
print("\nScanning...\n")
for port in range(start_port, end_port + 1):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(0.5)
    result = sock.connect_ex((host, port))
    if result == 0:
```

```
    print(f'Port {port} : OPEN")
else:
    print(f'Port {port} : CLOSED")
sock.close()
print("\nScanning Completed.")
```

6.Execution and Output Discussion:

INPUT:

Enter Target Host: scanme.nmap.org

Enter Starting Port: 20

Enter Ending Port: 30

OUTPUT:

Scanning...

Port 20 : CLOSED

Port 21 : CLOSED

Port 22 : OPEN

Port 23 : CLOSED

Port 24 : CLOSED

Port 25 : CLOSED

Port 26 : CLOSED

Port 27 : CLOSED

Port 28 : CLOSED

Port 29 : CLOSED

Port 30 : CLOSED

Scanning Completed.

The program successfully scanned the specified port range and identified whether each port on the target host was open or closed.

7. Conclusion:

The experiment successfully demonstrated port scanning using Python. It provided practical knowledge of identifying open network ports, an essential task in penetration testing and network security.

EXPERIMENT NO.-08

1. Aim of the Experiment:

To interact with the VirusTotal API using Python to retrieve security information about a file hash.

2. Objective:

- To understand the use of security-related APIs.
- To interact with the VirusTotal API using Python.
- To retrieve malware analysis information for a file hash.
- To gain practical experience in API-based security analysis.

3. Problem Statement:

Develop a Python program that accepts a VirusTotal API key and a file hash as input, queries the VirusTotal API, and displays the security analysis results.

4. Algorithm:

Step 1: Start.

Step 2: Import the required libraries.

Step 3: Read the VirusTotal API key.

Step 4: Read the file hash from the user.

Step 5: Send a request to the VirusTotal API.

Step 6: Display the analysis results.

Step 7: Stop.

5. Code Implementation:

```
!pip install requests
import requests
print("===== VirusTotal API =====")
api_key = input("Enter your VirusTotal API Key: ")
file_hash = input("Enter File Hash (MD5/SHA-1/SHA-256): ")
url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
headers = {
    "x-apikey": api_key
}
response = requests.get(url, headers=headers)
if response.status_code == 200:
    data = response.json()
```

```
stats = data["data"]["attributes"]["last_analysis_stats"]
print("\nAnalysis Result")
print("-----")
print("Malicious :", stats["malicious"])
print("Suspicious:", stats["suspicious"])
print("Harmless :", stats["harmless"])
print("Undetected:", stats["undetected"])
else:
    print("Error:", response.status_code)
    print("Invalid API Key or File Hash.")
```

6.Execution and Output Discussion:

INPUT:

Enter your VirusTotal API Key:

d906c5593af00f1e9bf5ca09b2b991739d8ae5911044d43d4550e2dddb5fb
25d

Enter File Hash (MD5/SHA-1/SHA-256):

44d88612fea8a8f36de82e1278abb02f

OUTPUT:

Analysis Result:

Malicious : 65

Suspicious: 0

Harmless : 0

Undetected: 2

The program successfully connected to the VirusTotal API and retrieved the security analysis report for the given file hash.

7. Conclusion:

The experiment successfully demonstrated how to interact with the VirusTotal API using Python. It provided practical experience in retrieving malware analysis information through a security-related API.

1. Aim of the Experiment:

To perform basic static malware analysis by identifying a file's signature and extracting readable strings using Python.

2. Objective:

- To understand basic static malware analysis.
- To identify file signatures.
- To extract readable strings from a file.
- To gain practical experience in malware analysis using Python.

3. Problem Statement:

Develop a Python program that accepts a file name as input, identifies its file signature, and extracts readable strings from the file without executing it.

4. Algorithm:

Step 1: Start.

Step 2: Read the file name from the user.

Step 3: Open the file in binary mode.

Step 4: Read the file signature (first four bytes).

Step 5: Extract readable strings from the file.

Step 6: Display the file signature and extracted strings.

Step 7: Stop.

5. Code Implementation:

```
import os
import re
print("===== STATIC MALWARE ANALYSIS =====")
filename = input("Enter file name: ")
if os.path.exists(filename):
    with open(filename, "rb") as file:
        data = file.read()
        print("\nFile Signature:", data[:4].hex().upper())
        print("\nReadable Strings:\n")
        strings = re.findall(rb"[ -~]{4,}", data)
        if strings:
            for s in strings[:20]:    # Display first 20 strings only
```

```
        print(s.decode(errors="ignore"))
    else:
        print("No readable strings found.")
else:
    print("File does not exist.")
```

6.Execution and Output Discussion:

INPUT:

Enter file name: NOV working days.xlsx

OUTPUT:

File Signature: 504B0304

Readable Strings: [Content_Types].xml |v~6}qFz,BKPNJ=IB1"o^
_rels/.rels b"gi4Mox/8q"Kxl/_rels/workbook.xml.rels nK|q
xl/workbook.xmlY?7`Z;}[?^tY^^,mDgiBq= e';Cc>xl/theme/theme1.xml

The program successfully identified the file signature and extracted readable strings from the file, demonstrating basic static malware analysis techniques.

7. Conclusion:

The experiment successfully demonstrated basic static malware analysis using Python by performing file signature analysis and string extraction without executing the file.